Knowledge Distillation and Deployment of a Lightweight Command-Parsing Model on a Local Cloud

CS3081 Cloud Computing, mini project report

Name [YOUR NAME]
Roll number [ROLL NUMBER]
Repository https://github.com/Sameer6556/cc (assessed commit: tag `final`)
Model releases model-v1, model-v2

Overview, architecture and how to read this report

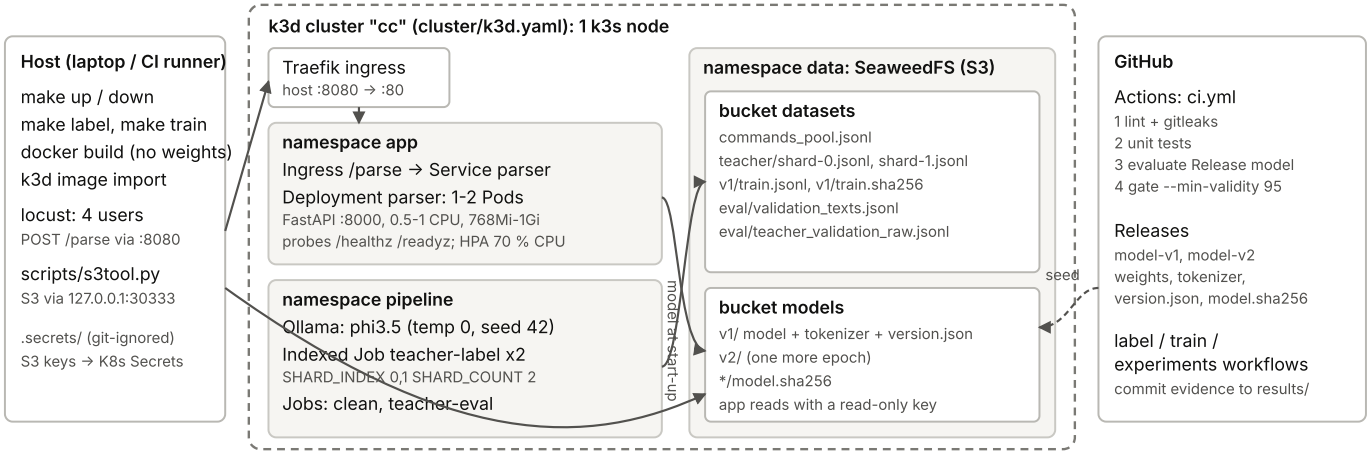


Figure 1. System architecture. Everything runs in one k3d cluster created from `cluster/k3d.yaml`; the only things outside it are the host tools and GitHub (CI and the model Releases).

The teacher (phi3.5, 3.8 B parameters, served by Ollama) labels the 3,000 pool commands with a four-key JSON frame; `clean.py` filters the replies into a training set; Pythia-70M is fine-tuned on the first 1,500 records (sequence-level distillation); the student is served by a FastAPI service behind a Traefik Ingress with an HPA. All long-running experiments were executed with the repository's own Makefile targets and scripts on GitHub-hosted runners (Ubuntu 24.04, 4 vCPU, 16 GiB RAM), from a fresh clone each time, by the workflows `label`, `train` and `experiments`, which commit their evidence to `results/`; the same commands run unchanged on a laptop (see README).

M measured: read from the file named in the caption or row A assumption or design choice, not a measurement

Task 1. Object storage and parallel teacher labelling

Architecture. The cluster is a single k3s node; Traefik is published on host port 8080 and the S3 port of SeaweedFS on 127.0.0.1:30333 only. SeaweedFS 4.47 runs as one `weed server -s3` Pod in namespace `data` on a 5 GiB volume. S3 credentials are generated at deploy time (`scripts/secrets.sh`) straight into Kubernetes Secrets: an admin identity for the pipeline and a read-only identity limited to the `models` bucket for the service (checked: GET/LIST on models allowed; GET on datasets, PUT and DELETE denied). `make up` creates the buckets `datasets` and `models` and uploads `commands_pool.jsonl`. The teacher runs inside the cluster (Ollama 0.40.9, namespace `pipeline`, phi3.5 pulled by an init container) with temperature 0 and seed 42 hard-coded in `worker/teacher.py`, plus `num_predict` 128 and `num_ctx` 2048.

Prompt (`prompts/teacher_prompt.txt`). It restates the four keys, gives a one-line definition per intent, the three slot rules, and two conventions of the data (drop "next"; parts of the day and words such as "near me" are not slots), followed by 13 examples I wrote. None of the 13 commands occurs in the pool or the validation set (checked by script), so no hand-made label can leak into the training data.

Sharding. Worker k of K owns the commands whose 0-based line number n satisfies $n \bmod K = k$. The workers are one Kubernetes *Indexed Job* with `completions = parallelism = 2`; each Pod receives its completion index through the downward API as `SHARD_INDEX`, and `SHARD_COUNT=2`. The assignment is a pure function of the line number, so the shards are disjoint and together cover every line without any coordination, locking or shared state; each worker writes only its own object `datasets/teacher/shard-k.jsonl`, so there are no write conflicts. Round-robin (rather than contiguous blocks) keeps the shards balanced even if command length drifts through the file. A worker uploads its partial object every 25 replies and, when restarted, skips the ids already present, so a Pod failure costs at most 25 teacher calls. Ollama decodes the two workers' requests together (`OLLAMA_NUM_PARALLEL=2`). The trade-off of a static split is that a slow worker cannot hand work to a fast one.

clean.py outcome (each record counted once, first rule that applies)	records
input records (both shards)	PENDING
(a) reply is not valid JSON	PENDING
(b) keys not exactly intent, date, time, place	PENDING
(c) intent not one of the eight	PENDING
(d) slot value neither null nor text of the command	PENDING
(e) duplicate command (smallest id kept)	PENDING
kept	PENDING

M results/task1/clean_stats.json. input = (a)+(b)+(c)+(d)+(e)+kept is asserted in clean.py.

worker	replies	wall time	s / command	reply tokens
PENDING				

M results/task1/worker-shard-*.log (both Pods ran concurrently: workers_running.txt).

Coverage (every id in exactly one shard, in the shard its line number dictates): **PENDING** (coverage.txt). Re-running `clean.py` outside the cluster reproduces `train.sha256`: **PENDING** (clean_reproduce.txt). SHA-256 of `datasets/v1/train.jsonl`: **PENDING**. Labelling wall time: **PENDING**.

Cleaning decisions A. A reply is parsed after trimming surrounding whitespace (as `score.py` does); a repeated key counts as (b); NaN is invalid JSON; rule (d) compares whole words, case- and space-insensitively, so "mon" does not match "monday" and an empty string never matches. Rules (a)-(d) are applied first, then duplicates among the survivors. Each kept line stores the exact `teacher_raw` reply and a `target`: the same object re-serialised with the key order intent, date, time, place (values unchanged; **PENDING** of **PENDING** kept replies were already byte-identical to it). **Limitations / improvements:** rule (d) checks form, not meaning, so a wrong-but-present span is kept; with only 4 vCPU the teacher is compute-bound, so more workers would not help; a work queue (or a Job per chunk) would remove the static-split imbalance, and a second teacher pass on rejected commands could recover data.

Task 2. Training the student (sequence-level distillation)

training/config.yaml	value
seed (fixed in code via set_seed before any randomness)	42
learning rate (AdamW, 5 % warm-up, linear decay)	0.0003
batch size	16
epochs: v1 / v2	10 / 11 (v2 = one more epoch)
maximum sequence length	128 tokens (no example truncated: **PENDING**)
libraries	Python 3.12, torch 2.14.1 (CPU), transformers 5.19.0, tokenizers 0.23.3, safetensors 0.8.0
records	first **PENDING** of train.jsonl (**PENDING** to **PENDING**)
base model	EleutherAI/pythia-70m @ **PENDING**

M config and results/task2/train_v1.log. Training time v1 **PENDING**, v2 **PENDING** on 4 vCPU; `make train SUBSET=50` ran to the end in **PENDING** (train_subset50.log).

Loss chart: **PENDING**

Figure 2. Mean per-token loss on the reply tokens, per epoch (training/loss.csv, loss_v2.csv) M

Implementation. Each example is `Command: <text>\nJSON: <target><|endoftext|>`. The loss is computed only on the reply and the end-of-text token (prompt tokens are masked with -100), so the student learns p(reply | command) and learns to stop. The prompt format, greedy decoding (stop at end-of-text or newline, `MAX_NEW_TOKENS=64`) and validation live in one module, `service/parser_core.py`, used by training, offline evaluation and the service, so the scores below are produced exactly the way the service answers. Training runs in deterministic mode; two runs on the same machine produced byte-identical weights. Both versions were uploaded to `s3://models/v1/` and `v2/` (results/task2/s3_listing.txt) and published as Releases whose files verify against `models/v1/model.sha256` with `sha256sum -c` (results/task2/release_v1_sha256_check.txt). Weights: **PENDING**.

model (validation.jsonl, 100 commands)	JSON valid	intent	date	time	place	full frame
teacher phi3.5 (same prompt and settings as Task 1)	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
Pythia-70M, untrained (same prompt and decoding)	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
student v1 (10 epochs)	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
student v2 (11 epochs)	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING

M score.py reports: results/teacher_validation.json, results/task2/untrained_validation.json, student_v1_validation.json, student_v2_validation.json (raw outputs next to them). Student v1 full frame = **PENDING** of the teacher's.

What changed. The untrained model does not produce the frame at all; after distillation the student emits a schema-valid frame for almost every command, and its errors are concentrated in the slots, where it has to copy an exact span. Because the student is trained only on the teacher's replies, the teacher's full-frame accuracy is effectively its ceiling, and the teacher's systematic deviations from the SLURP conventions are inherited.

Sequence-level vs logit-based distillation
(a) Sequence-level KD (Kim and Rush, 2016), used here, trains the student with ordinary cross-entropy on the teacher's *generated output sequence*: the teacher's (approximate) mode, decoded greedily, is treated as the hard target ŷ, and the student maximises log $p_S(\hat{y} \mid x)$. **(b) Logit-based (word-level) KD** (Hinton et al., 2015) instead trains on the teacher's *full next-token probability distribution* at every position, minimising $\mathrm{KL}(p_T(\cdot \mid x, y_{<t}) \,||\, p_S(\cdot \mid x, y_{<t}))$, usually with a temperature, so the student also learns how the teacher spreads probability over the alternatives (the "dark knowledge"). **(c) It is not possible here** for two independent reasons: Ollama's API returns the generated text, not the per-position distribution over the whole vocabulary; and even with logits, phi3.5 (Llama-style SentencePiece, 32,064 tokens) and Pythia (GPT-NeoX BPE, 50,304 tokens) tokenise text differently, so their distributions are over different vocabularies and different token boundaries and cannot be compared position by position without an alignment method. **(d) Limitation of what I did:** the student sees one hard answer per command, so it copies the teacher's mistakes with full confidence (for example its slot boundaries, which differ from the SLURP labels used for scoring), and it gets no signal about which commands the teacher was unsure of; the filter in Task 1 removes malformed replies but cannot remove fluent, wrong ones.

Task 3. The inference service in a container

Implementation. FastAPI (`service/app.py`), port 8000. `POST /parse` runs greedy generation, validates the output against `schema.json` with `jsonschema` (exactly the four keys, one of the eight intents, string-or-null slots, NaN rejected) and returns either HTTP 200 with only the object or HTTP 500 `{"error": "invalid_model_output"}`; both carry `X-Model-Version` and `X-Served-By` (the Pod name from the downward API). `/healthz` answers 200 while the process lives; `/readyz` is 503 until a background thread has downloaded every object under `MODEL_URI` from `S3_ENDPOINT`, verified each file against `model.sha256`, loaded the model and run one warm-up generation. Configuration comes from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT` and `MAX_NEW_TOKENS`; credentials only from the Secret; `FAULT_INJECT=1` replaces the output with `not json` before validation. One generation runs at a time per Pod (a lock), and torch uses as many threads as the container's CPU limit (read from the cgroup). The image (`service/Dockerfile`) installs the CPU-only torch wheel, then every other package from a fully pinned lock with `--no-deps` and `pip check`; it runs as a non-root user and contains no weights.

check	result	evidence (results/task3/)
image size (limit 2 GB)	**PENDING**	image.txt
model files inside the image	**PENDING**	no_weights_in_image.txt
/readyz 503 while loading, then 200; /healthz 200 throughout	**PENDING**	readiness.log, endpoints.txt
50 sequential requests at `--cpus 1 --memory 1g`	**PENDING**, codes **PENDING**; median **PENDING**, max **PENDING**	sequential_50.txt
peak memory (cgroup `memory.peak`), OOM-killed	**PENDING** of 1,024 MiB; OOMKilled=**PENDING**	memory_peak.txt, docker_stats.log
`FAULT_INJECT=1`	**PENDING**	fault_inject.txt

M all rows; non-200 codes in the 50-request run are invalid model outputs, which the service correctly refuses.

Limitations / improvements. torch dominates the image size and start-up time; exporting the model to ONNX Runtime would shrink both. One request at a time per Pod caps throughput at one CPU's worth of decoding; batching concurrent requests would raise it. Invalid outputs are refused, not repaired; constrained decoding against the JSON schema would make every answer valid by construction.

Task 4. Kubernetes deployment, scaling and reliability

Implementation (`k8s/app/parser.yaml`). Namespace `app`: a Deployment starting at 2 replicas (requests 500m CPU / 768 Mi, limits 1 CPU / 1 Gi; readiness probe `/readyz` every 2 s, liveness `/healthz` every 10 s with 6 allowed failures; rolling update `maxSurge 1, maxUnavailable 0`; `preStop` sleep 5 s so Traefik drops a terminating Pod before it stops; graceful shutdown 20 s), a ClusterIP Service, a Traefik Ingress for `/parse` only, and an `autoscaling/v2` HPA (1 to 2 replicas, 70 % average CPU of the request, scale-down window shortened to 60 s so it can be observed). The scripts in `scripts/experiments/` produce every row below.

experiment	measured result	evidence (results/task4/)
/parse through the Ingress	PENDING	basics/ingress_curl.txt, resources.txt
no ready endpoints while the model loads	PENDING timeline samples (0.5 s apart) with 0 ready endpoints after deleting all Pods	coldstart/run*_endpoints.log
HPA under load (4 users)	replicas PENDING → PENDING → PENDING; PENDING failed of PENDING	hpa/timeline.log, describe_hpa.txt
rolling update v1→v2, then `rollout undo`, under load	PENDING failed (PENDING of PENDING); versions served PENDING; after rollback PENDING of 5 answers say v1	rolling/rolling.log, version_timeline.log, after_rollback.txt
delete one Pod under load	back to 2 Ready after PENDING; PENDING failed (PENDING of PENDING)	recovery/recovery.log, timeline.log
cold start: delete all Pods, 3 runs	PENDING; **median** PENDING (Pod creation to Ready); logs show the download from `s3://models/v1/`	coldstart/summary.txt, run*_*.log
load balancing, exactly 2 Ready replicas	successful requests per replica: PENDING (each must be ≥ 30 %)	loadbalance/loadbalance.log

M all rows. For the recovery and load-balancing runs the HPA minimum was held at 2 so that two replicas existed; it was set back to 1 afterwards.

Limitations. A The cluster is one k3s node in one container on one machine: the experiments show recovery from a lost *Pod*, not from a lost node, disk or machine; there is no fault tolerance across machines, and SeaweedFS is a single replica on a single volume. The HPA reacts to metrics-server samples (15 s) plus a stabilisation window, so it lags short bursts; and the replica count is capped at 2 by the assignment. **Improvements:** 3+ nodes (k3d agents) with pod anti-affinity, a PodDisruptionBudget, SeaweedFS replication, a startup probe instead of a long liveness delay, and an HPA on request latency rather than CPU.

Task 5. Subnetting and network access control

subnet	network / prefix	usable range	broadcast	hosts
app (1,000)	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public (250)	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data (100)	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management (20)	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

Working in network/subnets.md: largest first, smallest prefix with $2^{32-p}-2 \geq$ need (one bit smaller gives 510, 126, 62, 14: too few); each block starts right after the previous one. Verified with Python ipaddress in tests/test_network.py.

dir	rule	match	action
in	50	all, 198.51.100.0/24	deny
in	100 / 110	tcp 80 / tcp 443, 0.0.0.0/0	allow
in	120	tcp 22, 10.20.5.128/27 (management)	allow
in	130	tcp 1024-65535, 10.20.0.0/22 (app replies)	allow
out	100	tcp 8000, 10.20.0.0/22 (app)	allow
out	110	tcp 1024-65535, 0.0.0.0/0 (client replies)	allow
both	32766	all, 0.0.0.0/0	deny

M nacl_sim.py on packets_sample.jsonl: 6 / 6 correct; 16 / 16 on my edge cases (results/network/nacl_check.txt).

Why stateless rules need reply rules, and why order matters. A NACL keeps no connection state: every packet, including a reply, is evaluated on its own. When a client opens a connection to the load balancer on 443, the answer leaves the subnet towards the client's ephemeral port (1024-65535); without outbound rule 110 every response would be dropped. Likewise the load balancer's request to the app goes out on TCP 8000, but the app's answer arrives on one of the load balancer's ephemeral ports and needs inbound rule 130 (a stateful security group would allow both automatically). Rules are checked from the lowest number and the first match wins. A packet from 198.51.100.9 to port 443 matches both the deny and the "443 from anywhere" allow, so the deny only works if its number is lower than the allow it must override: renumbering it from 50 to 140 lets that packet in (results/network/nacl_wrong_order_check.txt, 5 / 6). Limitation: a stateless NACL cannot tell a reply from a new connection, so outbound rule 110 also permits new connections to any high port.

Task 6. Reproducibility, testing and CI/CD

Reproducibility. From a fresh clone the whole system comes up with three commands (`git clone`, `cd cc`, `make up`): cluster, Secrets, SeaweedFS, buckets and dataset, model Releases downloaded, verified and uploaded, image built and imported, app deployed, smoke request. `make down` deletes the cluster. The `experiments` workflow runs exactly this on a clean runner: `make up` took PENDING (results/task6/make_up.log) and `make down` left PENDING (make_down.log). Python dependencies are pinned (service image: full lock); the cluster image, SeaweedFS and Ollama tags are pinned.

ci.yml stage (in order)	what it does
1. lint + secret scan	`ruff check .`; gitleaks 8.30.1 over the full Git history (must report nothing)
2. unit tests	54 pytest tests: clean.py rules, sharding, service API and headers, schema validation, subnets, NACL
3. model evaluation	downloads the Release `model-v1`, checks `sha256sum -c model.sha256`, predicts validation.jsonl, runs score.py
4. quality gate	`score.py --min-validity 95`; fails the pipeline below 95 % JSON validity

M passing run: **PENDING**; run with the untrained Pythia-70M failing at the gate: **PENDING**.

The AI tools used, and how their output was checked, are described in `AI_USE.md`. **Limitation:** CI evaluates on the 100 validation commands only and cannot reproduce the multi-hour labelling; **improvement:** a nightly job that re-runs `make up` and the load test, and a held-out split of the pool.

Task 7. Performance, cost and review

standard load test (4 users, 2 min, via Ingress)	value
requests / failed	**PENDING** / **PENDING**
error rate	**PENDING**
p50 latency	**PENDING**
p95 latency	**PENDING**
throughput	**PENDING**
mean replicas during the run	**PENDING**
cold start, median of 3 (Task 4)	**PENDING**

M results/loadtest_stats.csv, results/task7/summary.json, timeline.log.

Pod (kubectl top, every 10 s)	CPU mean / max	memory mean / max	n
PENDING			

M results/task7/top_pods.log. metrics-server reports a 15 s window, so the first sample after the load starts still shows idle CPU.

> **Cost per 1,000 requests** (data/rate_card.json: INR 3.50 per vCPU-hour, INR 0.45 per GiB-hour; each replica billed for its limits, 1 vCPU and 1 GiB):
> 1. **PENDING**

SLO. ***PENDING*** over the window ***PENDING***. Error budget = 1 % of **PENDING** requests = **PENDING** failed requests; used: **PENDING** failed = **PENDING** of the budget (SLO **PENDING**). A The cost counts only the parser replicas at their limits (not SeaweedFS, Traefik or the control plane) and assumes the measured throughput is sustained for the whole hour.

component or concern	evidence (a measured number)	limitation found	improvement proposed
Data and storage	**PENDING** of **PENDING** teacher replies kept; train.jsonl checksum reproduced	rule (d) keeps fluent but wrong spans; single SeaweedFS replica	second teacher pass on rejects; replicated object store
Model quality	student full frame **PENDING**, JSON validity **PENDING**	copies the teacher's slot-boundary errors; 70 M parameters	logit KD with a same-tokenizer teacher; constrained decoding
Latency and throughput	p95 **PENDING**, **PENDING**	one request at a time per Pod; greedy decoding on 1 CPU	request batching; ONNX Runtime / int8
Resource utilisation	peak memory **PENDING** of 1 GiB at 1 CPU	CPU saturates while memory is mostly idle	smaller memory limit (cheaper); more replicas per node
Reliability and recovery	rolling update **PENDING** failed; cold start median **PENDING**	single node: no machine-level fault tolerance	multi-node cluster, anti-affinity, PodDisruptionBudget
Security and cost	gitleaks: 0 findings; INR **PENDING** per 1,000 requests	S3 over plain HTTP inside the cluster; replicas billed at full limits when idle	TLS + NetworkPolicies; scale to zero off-peak

Review. M evidence column; the other columns are my assessment.

References

1. Y. Kim, A. M. Rush. Sequence-Level Knowledge Distillation. EMNLP 2016. https://aclanthology.org/D16-1139/
2. G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. arXiv:1503.02531, 2015.
3. Y.-C. Chen et al. Distilling Knowledge Learned in BERT for Text Generation. ACL 2020.
4. E. Bastianelli et al. SLURP: A Spoken Language Understanding Resource Package. EMNLP 2020 (CC BY 4.0).
5. S. Biderman et al. Pythia: A Suite for Analyzing Large Language Models. ICML 2023; EleutherAI/pythia-70m model card.
6. Microsoft. Phi-3 Technical Report (Phi-3.5-mini-instruct); Ollama library phi3.5.
7. Hugging Face Transformers and TRL documentation (causal LM fine-tuning; GKD/distillation trainer).
8. k3d and K3s documentation; SeaweedFS (S3 API); Kubernetes docs: Deployments, Services, Ingress, probes, HPA, Secrets, Indexed Jobs.
9. Locust; gitleaks; AWS VPC User Guide, Network ACLs; Google SRE Book, Service Level Objectives.

Appendix A. Measured results vs assumptions

Measured (each number in the body is read at build time by `report/build_report.py` from these files): results/task1 (labelling, cleaning, coverage), datasets/v1/train.sha256, training/loss*.csv, results/teacher_validation.json, results/task2 (training logs, evaluations, S3 listing, Release checks), results/task3, results/task4, results/network, results/task6, results/loadtest_* and results/task7.

Assumptions and design choices (not measurements): the rate card prices are hypothetical; cost counts only the parser replicas at their limits and assumes the measured throughput holds for an hour; the SLO target (99 %) and window (one standard run) are my choices; the HPA scale-down window was shortened to 60 s for observability; the HPA minimum was raised to 2 during the recovery and load-balancing runs; validation scores are optimistic for the teacher because the prompt's conventions were chosen after looking at the kind of labels in validation.jsonl (no validation command or label is in the prompt); a GitHub-hosted runner (4 vCPU, 16 GiB) stands in for a student laptop.

Appendix B. Reproducing every number

`make up`; `make label` (Task 1); `make train`, `make train MODEL_VERSION=v2`, `make eval`, `make eval-untrained` (Task 2); `bash scripts/experiments/run_all.sh` (Tasks 3, 4, 7); `python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check` (Task 5); `make lint test` (Task 6); `python report/build_report.py && node report/render.mjs` (this report).